

ALIGNED LAYER

Security Assessment

June 2026



Contents

About FuzzingLabs	3
Executive Summary	4
Goals	4
Summary of Findings	5
Project Summary	7
About Aligned Layer	7
Scope	8
Audit Timeline	9
Threat Model	10
Findings	11
I. Ownership can be renounced, permanently disabling the airdrop	11
II. transferFrom used without SafeERC20	12
III. Centralization and operational dependencies	15
IV. Compiler version outdated, known bug fixed in 0.8.34	16
Conclusion	17
Disclaimer	17

About FuzzingLabs

Founded in 2021 and headquartered in Paris, FuzzingLabs is a cybersecurity startup specializing in vulnerability research, fuzzing, and blockchain security. We combine cutting-edge research with hands-on expertise to secure some of the most critical components in the blockchain ecosystem.

At FuzzingLabs, we aim to uncover and mitigate vulnerabilities before they can be exploited. Over the past year, our tools and methodologies have identified hundreds of vulnerabilities in essential blockchain components, such as RPC libraries, cryptographic systems, compilers, and smart contracts. We collaborate with leading protocols and foundations to deliver open-source security tools, continuous audits, and comprehensive fuzzing services that help secure the future of blockchain technology.

If you're interested, we have a blog available at fuzzinglabs.com and an X account [@FuzzingLabs](https://twitter.com/FuzzingLabs). You can also contact us at contact@fuzzinglabs.com.

Executive Summary

Goals

The primary goal of this audit is to assess [ClaimableAirdrop.sol](#) and verify that the contract behaves correctly under all anticipated conditions. The objectives are structured into two phases:

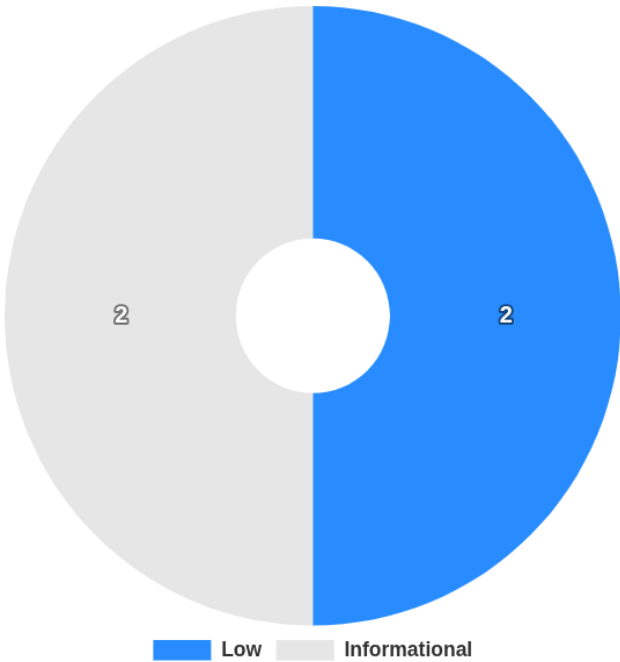
1. **Static Analysis:**

- Perform automated analysis of the contract using Slither and Aderyn to detect common vulnerability patterns, insecure coding practices, and deviations from best practices.
- Triage tool output manually to confirm or discard each alert, and identify areas warranting deeper manual scrutiny.

2. **Manual Review:**

- Conduct a thorough line-by-line examination of the contract to identify logical bugs, access control weaknesses, and vulnerabilities that automated tools may miss.
- Verify the correctness of the Merkle verification scheme, double-claim prevention, vesting enforcement, reentrancy protection, and ERC-20 interaction against both the specification and the in-scope token.

Summary of Findings



ID	Title	Target	Rating
1	Ownership can be renounced, permanently disabling the airdrop	ClaimableAirdrop.sol	Low
2	transferFrom used without SafeERC20	ClaimableAirdrop.sol	Low
3	Centralization and operational dependencies	ClaimableAirdrop.sol	Info
4	Compiler version outdated, known bug fixed in 0.8.34	ClaimableAirdrop.sol	Info

Project Summary

About Aligned Layer

Aligned Layer is a decentralized network designed to provide fast, efficient, and low-cost verification of zero-knowledge (ZK) and validity proofs on the Ethereum blockchain. It seeks to overcome the limitations of current blockchain verification systems, which are often slow and expensive due to the need for nodes to re-execute each transaction. By offloading the computation off-chain, Aligned Layer enables faster proof verification and significantly reduces costs.

The platform operates in two modes: **fast mode** and **aggregation mode**. In **fast mode**, a subset of Ethereum's validators (operators) verify proofs in parallel, posting the results to Ethereum only after reaching a two-thirds consensus. This approach allows Aligned Layer to achieve verification speeds of over 1,000 proofs per second, far outpacing Ethereum's native capabilities. In **aggregation mode**, the system further compresses proofs for even greater efficiency at scale.

Aligned Layer is particularly useful for applications requiring large-scale computations with the assurance that they are verified in a trustless manner. It supports various proof systems, making it versatile for developers working on ZK technology, including zk-rollups, identity protocols, and decentralized applications that demand high throughput and low latency.

The goal of Aligned Layer is to accelerate Ethereum's roadmap, supporting the adoption of verifiable computation and making ZK verification accessible and affordable to a wider range of developers.

Scope

This assessment was conducted on [ClaimableAirdrop.sol](#) at the commit [f5aed29](#)

The audit focused on the following components, identified as essential for contract security:

- **Claim Logic:** The core `claim` and `claimBatch` functions, including Merkle proof verification, double-claim prevention, and vesting stage enforcement.
- **Access Control:** Ownership model via `Ownable2StepUpgradeable`, protection of privileged functions, and the ability to update the Merkle root and extend the claim period.
- **ERC-20 Interaction:** Token transfer mechanics between `tokenDistributor` and claimants, and the trust assumptions placed on `tokenProxy`.
- **Reentrancy:** Correctness of the checks-effects-interactions pattern and the `nonReentrant` guard across both entry points.
- **Upgradeability:** Proxy pattern, storage layout safety, and initialization protection.
- **Centralization & Operational Risk:** Owner privileges, distributor liveness dependency, and deployment procedure safety.

[AlignedToken.sol](#) was reviewed as context only to understand the in-scope token's behavior and is not itself in scope for findings. Off-chain components (Merkle tree generator, deployment scripts) are out of scope.

Audit Timeline

This assessment was conducted over a total of 1 man-day by one engineer, structured as follows:

1. Static Analysis:

- A static analysis of [ClaimableAirdrop.sol](#) was performed using Slither and Aderyn, focusing on potential vulnerabilities, security flaws, and code patterns that may expose the contract to threats. Tool output was triaged manually to confirm or discard each alert.

2. Manual Review:

- A thorough line-by-line review of [ClaimableAirdrop.sol](#) was conducted, covering access control, claim and vesting logic, Merkle verification and replay protection, double-claim prevention, reentrancy, arithmetic, ERC-20 interaction, and upgradeability.

This structured approach ensured a comprehensive security review of [ClaimableAirdrop.sol](#), covering both automated detection of common vulnerability patterns and deep manual analysis of the contract's logic, access control, and interaction surface.

Threat Model

Based on the attack surface presented by [ClaimableAirdrop.sol](#), we constructed a threat model identifying the critical risks associated with the contract's role as a token distribution mechanism. By analyzing the interactions between claimants, the owner, the `tokenDistributor`, and the underlying token proxy, we highlighted potential vulnerabilities in access control, claim logic, and operational configuration. The threat model provides a structured view of these risks, facilitating focused security assessments and suggesting mitigations tailored to each component's role.

The following considerations were taken into account when creating the threat model:

Claim Logic

- Ability for a user to claim more tokens than entitled, claim the same allocation twice, or claim a stage not yet vested.
- Ability to forge or replay a Merkle proof to claim without a valid entitlement.

Access Control

- Unauthorized access to privileged functions such as root updates and claim period extension.
- Permanent loss of admin control through ownership renouncement or transfer to an uncontrolled address.

Token Interaction

- Behavior of the contract under non-standard or upgraded `tokenProxy` implementations.
- Impact of a transfer failure on claim state and user recoverability.

Operational & Centralization

- Owner ability to freeze or disrupt claims through pausing or root manipulation.
- Sufficiency of `tokenDistributor` balance and allowance throughout the claim period.
- Safety of upgrade and deployment procedures against front-running or misconfiguration.

Findings

I. Ownership can be renounced, permanently disabling the airdrop

Rating	Low
ID	FL-AL-1
Category	Access Control
Target	ClaimableAirdrop.sol
Commit	f5aed29

Description

`AlignedToken` overrides `renounceOwnership()` to revert, `ClaimableAirdrop` does not. `Ownable2StepUpgradeable` only overrides `transferOwnership` and `_transferOwnership`, it leaves `renounceOwnership` from `OwnableUpgradeable` intact, meaning the owner can set ownership to `address(0)` in a single step with no confirmation.

Impact

If ownership is renounced, `updateMerkleRoot`, `extendClaimPeriod`, `pause`, and `unpause` become permanently uncallable, freezing claims if paused, or leaving the contract unpausable if active. Owner-only action (hence Low), but permanent.

Recommendations

Override `renounceOwnership()` to revert, matching the token.

```
...  
/// @notice Prevents the owner from renouncing ownership.  
function renounceOwnership() public view override onlyOwner {  
    revert("Cannot renounce ownership");  
}  
...
```

II. transferFrom used without SafeERC20

Rating	Low
ID	FL-AL-2
Category	Best Practices
Target	ClaimableAirdrop.sol
Commit	f5aed29

Description

Token transfers use `IERC20.transferFrom` with a boolean return check. This is **not insecure with the in-scope token**: `AlignedToken` is a standard OpenZeppelin ERC-20 that returns `true` and reverts on failure, so the check is correct and a failed transfer safely reverts the whole claim without stranding the leaf. It is, however, not best practice. The pattern does not tolerate non-standard tokens (e.g. those returning no data, where the bool-decode would revert and brick all claims), and `tokenProxy` is itself an upgradeable proxy whose implementation could change.

Impact

None under the current token. Latent compatibility/robustness risk if the token ever deviates from the standard ERC-20 return convention.

Recommendations

Use `SafeERC20.safeTransferFrom`:

```
...
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

using SafeERC20 for IERC20;

// claim:
IERC20(tokenProxy).safeTransferFrom(tokenDistributor, msg.sender, amount);

// claimBatch:
IERC20(tokenProxy).safeTransferFrom(tokenDistributor, msg.sender, totalClaimable);
// (the bool `success` variable and its require are no longer needed)
...
```

III. Centralization and operational dependencies

Rating	Informational
ID	FL-AL-3
Category	Centralization
Target	ClaimableAirdrop.sol
Commit	f5aed29

Description

The owner is fully trusted: it can swap the root (while paused), pause to freeze claims, or extend the deadline. Claim liveness also depends on `tokenDistributor` keeping sufficient balance and an approval to this contract.

Impact

A compromised owner can disrupt the distribution. An unfunded/unapproved distributor causes claims to revert (the tx rolls back, including the `hasClaimed` write, so users can retry once funded).

Recommendations

Use a multisig/timelock owner, monitor the distributor's allowance/balance.

IV. Compiler version outdated, known bug fixed in 0.8.34

Rating	Informational
ID	FL-AL-4
Category	Code Quality
Target	ClaimableAirdrop.sol
Commit	f5aed29

Description

Solidity 0.8.34 (released February 2026) patches a high-severity IR pipeline bug present in 0.8.28-0.8.33, involving incorrect clearing of transient storage variables when compiled with `--via-ir`.

Impact

`ClaimableAirdrop` is not affected. It does not use transient storage, does not call `delete` on any state variable, and does not use `--via-ir`. No vulnerability is present in the current codebase.

Recommendations

Upgrade to `pragma solidity 0.8.35` (current latest stable) as a hygiene measure and to stay aligned with the upstream bugfix baseline.

Conclusion

Our audit of [ClaimableAirdrop.sol](#) found a well-structured implementation with no attacker-reachable vulnerabilities. The contract correctly handles Merkle proof verification, double-claim prevention, vesting enforcement, and reentrancy protection. The findings identified are limited to two low severity items and two informational notes, all of which are straightforward to address.

Disclaimer

This report is provided under the terms of the agreement between FuzzingLabs and the client. It is intended solely for the client's use and may not be shared or referenced without FuzzingLabs' prior written consent. The findings in this report are based on a point-in-time assessment and do not guarantee the absence of vulnerabilities or security flaws. FuzzingLabs cannot ensure future security as systems and threats evolve. We recommend continuous monitoring, independent assessments, and a bug bounty program.

This report is not financial, legal, or investment advice, and should not be used for decision-making regarding project involvement or investment. FuzzingLabs aims to help reduce risks, but we do not provide any guarantees regarding the complete security of the technology assessed.